

David_数据结构

David_数据结构

- 区间加乘线段树
- 二维线段树
- 李超线段树
- 主席树
- 并查集
- 带权并查集
- 普通莫队
- 带修莫队
- 回滚莫队

区间加乘线段树

```
1 //线段树，支持区间加与区间乘
2 struct Seg{
3     int n, mod;
4     vector<int> tr, lazyAdd, lazyMul;
5
6     Seg(int n, int mod) : n(n), mod(mod),
7         tr(n * 4 + 5),
8         lazyAdd(n * 4 + 5),
9         lazyMul(n * 4 + 5, 1) {}
10
11     void build(int p, int l, int r, const vector<int>& a){
12         if(l == r){
13             tr[p] = a[l] % mod;
14             return;
15         }
16         int mid = l + r >> 1;
17         build(p << 1, l, mid, a);
18         build(p << 1 | 1, mid + 1, r, a);
19         pushup(p);
20     }
21
22     void clear(){
23         fill(tr.begin(), tr.end(), 0);
24         fill(lazyAdd.begin(), lazyAdd.end(), 0);
25         fill(lazyMul.begin(), lazyMul.end(), 1);
26     }
27
28     void pushup(int p){
29         tr[p] = (tr[p << 1] + tr[p << 1 | 1]) % mod;
30     }
31
32     void apply(int p, int l, int r, int mul, int add){
33         tr[p] = (1LL * tr[p] * mul % mod + 1LL * (r - l + 1) * add % mod) % mod;
34         lazyMul[p] = 1LL * lazyMul[p] * mul % mod;
35         lazyAdd[p] = (1LL * lazyAdd[p] * mul % mod + add) % mod;
36     }
37
38     void pushdown(int p, int l, int r){
```

```

39     if(lazyMul[p] != 1 || lazyAdd[p] != 0){
40         int mid = l + r >> 1;
41         apply(p << 1, l, mid, lazyMul[p], lazyAdd[p]);
42         apply(p << 1 | 1, mid + 1, r, lazyMul[p], lazyAdd[p]);
43         lazyMul[p] = 1;
44         lazyAdd[p] = 0;
45     }
46 }
47
48 void updateMul(int p, int l, int r, int ql, int qr, int val){
49     if(ql <= l && r <= qr){
50         apply(p, l, r, val, 0);
51         return;
52     }
53     pushdown(p, l, r);
54     int mid = l + r >> 1;
55     if(ql <= mid) updateMul(p << 1, l, mid, ql, qr, val);
56     if(mid < qr) updateMul(p << 1 | 1, mid + 1, r, ql, qr, val);
57     pushup(p);
58 }
59
60 void updateAdd(int p, int l, int r, int ql, int qr, int val){
61     if(ql <= l && r <= qr){
62         apply(p, l, r, 1, val);
63         return;
64     }
65     pushdown(p, l, r);
66     int mid = l + r >> 1;
67     if(ql <= mid) updateAdd(p << 1, l, mid, ql, qr, val);
68     if(mid < qr) updateAdd(p << 1 | 1, mid + 1, r, ql, qr, val);
69     pushup(p);
70 }
71
72 int ask(int p, int l, int r, int ql, int qr){
73     if(ql <= l && r <= qr){
74         return tr[p];
75     }
76     pushdown(p, l, r);
77     int mid = l + r >> 1;
78     int res = 0;
79     if(ql <= mid) res = (res + ask(p << 1, l, mid, ql, qr)) % mod;
80     if(mid < qr) res = (res + ask(p << 1 | 1, mid + 1, r, ql, qr)) % mod;
81     return res;
82 }
83 };
84
85 //非递归线段树，适用于单点修改区间查询的问题，常数更小
86 struct Tree {
87     int sz; // 大于值域上限的 2 的幂
88     vector<int> tree;
89
90     // n: 值域右端点 (1-based)，实际会用到的最大下标
91     Tree(int n) {
92         sz = 1;
93         while (sz < n) sz <= 1;
94         tree.assign(sz * 2, 0); // 1-based 索引，叶子从 sz 开始
95     }
96
97     // 清空线段树（全部置零），供下一轮重用
98     void clear() {
99         fill(tree.begin(), tree.end(), 0);

```

```

100     }
101
102     // 单点更新: 忽略前三个无用参数, 只保留 tar, val
103     void update(int /*ind*/, int /*l*/, int /*r*/, int tar, int val) {
104         int p = tar + sz - 1;          // tar 是 1-based
105         tree[p] = val;
106         p >>= 1;
107         while (p) {
108             tree[p] = max(tree[p << 1], tree[p << 1 | 1]);
109             p >>= 1;
110         }
111     }
112
113     // 区间最大值查询: 忽略前三个无用参数, ql,qr 为 1-based
114     int query(int /*ind*/, int /*l*/, int /*r*/, int ql, int qr) {
115         if (ql > qr) return 0;
116         int l = ql + sz - 1;
117         int r = qr + sz - 1;
118         int ans = 0;
119         while (l <= r) {
120             if (l & 1) ans = max(ans, tree[l++]);
121             if (!(r & 1)) ans = max(ans, tree[r--]);
122             l >>= 1;
123             r >>= 1;
124         }
125         return ans;
126     }
127 };

```

二维线段树

```

1 // 二维线段树
2 struct Seg2D {
3     int n, m;
4     vector<vector<int>> tr, lazy;
5
6     Seg2D(int n, int m) : n(n), m(m) {
7         tr.assign(n * 4 + 5, vector<int>(m * 4 + 5, 0));
8         lazy.assign(n * 4 + 5, vector<int>(m * 4 + 5, 0));
9     }
10
11     void build(int p, int l, int r, const vector<vector<int>>& a) {
12         build_y(p, 1, 1, m, l, r, a);
13     }
14
15     // 构建Y轴
16     void build_y(int px, int py, int ly, int ry, int lx, int rx, const
vector<vector<int>>& a) {
17         if (ly == ry) {
18             if (lx == rx) {
19                 tr[px][py] = a[lx][ly];
20             } else {
21                 tr[px][py] = tr[px << 1][py] + tr[px << 1 | 1][py];
22             }
23             return;
24         }
25         int mid = ly + ry >> 1;
26         build_y(px, py << 1, ly, mid, lx, rx, a);
27         build_y(px, py << 1 | 1, mid + 1, ry, lx, rx, a);
28         pushup_y(px, py);

```

```

29     }
30
31     void build(int px, int lx, int rx, const vector<vector<int>>& a) {
32         if(lx == rx) {
33             build_y(px, 1, 1, m, lx, rx, a);
34             return;
35         }
36         int mid = lx + rx >> 1;
37         build(px << 1, lx, mid, a);
38         build(px << 1 | 1, mid + 1, rx, a);
39         build_y(px, 1, 1, m, lx, rx, a);
40     }
41
42     void clear() {
43         for(auto& row : tr) fill(row.begin(), row.end(), 0);
44         for(auto& row : lazy) fill(row.begin(), row.end(), 0);
45     }
46
47     void pushup_y(int px, int py) {
48         tr[px][py] = tr[px][py << 1] + tr[px][py << 1 | 1];
49     }
50
51     void pushdown_y(int px, int py, int ly, int ry) {
52         if(lazy[px][py]) {
53             int mid = ly + ry >> 1;
54             tr[px][py << 1] += (mid - ly + 1) * lazy[px][py];
55             tr[px][py << 1 | 1] += (ry - mid) * lazy[px][py];
56             lazy[px][py << 1] += lazy[px][py];
57             lazy[px][py << 1 | 1] += lazy[px][py];
58             lazy[px][py] = 0;
59         }
60     }
61
62     // 内层Y轴更新
63     void update_y(int px, int py, int ly, int ry, int qly, int qry, int val) {
64         if(qly <= ly && ry <= qry) {
65             tr[px][py] += (ry - ly + 1) * val;
66             lazy[px][py] += val;
67             return;
68         }
69         pushdown_y(px, py, ly, ry);
70         int mid = ly + ry >> 1;
71         if(qly <= mid) update_y(px, py << 1, ly, mid, qly, qry, val);
72         if(mid < qry) update_y(px, py << 1 | 1, mid + 1, ry, qly, qry, val);
73         pushup_y(px, py);
74     }
75
76     // 外层X轴更新
77     void update(int p, int l, int r, int ql, int qr, int qly, int qry, int val) {
78         if(ql <= l && r <= qr) {
79             update_y(p, 1, 1, m, qly, qry, val);
80             return;
81         }
82         int mid = l + r >> 1;
83         if(ql <= mid) update(p << 1, l, mid, ql, qr, qly, qry, val);
84         if(mid < qr) update(p << 1 | 1, mid + 1, r, ql, qr, qly, qry, val);
85         // 更新当前节点
86         for(int py = 1; py <= m * 4; py++) {
87             tr[p][py] = tr[p << 1][py] + tr[p << 1 | 1][py];
88         }
89     }

```

```

90
91 // 内层Y轴查询
92 int ask_y(int px, int py, int ly, int ry, int ql, int qry) {
93     if(ql <= ly && ry <= qry) {
94         return tr[px][py];
95     }
96     pushdown_y(px, py, ly, ry);
97     int mid = ly + ry >> 1;
98     int res = 0;
99     if(ql <= mid) res += ask_y(px, py << 1, ly, mid, ql, qry);
100    if(mid < qry) res += ask_y(px, py << 1 | 1, mid + 1, ry, ql, qry);
101    return res;
102 }
103
104 // 外层X轴查询
105 int ask(int p, int l, int r, int ql, int qr, int ql, int qry) {
106     if(ql <= l && r <= qr) {
107         return ask_y(p, l, l, r, ql, qry);
108     }
109     int mid = l + r >> 1;
110     int res = 0;
111     if(ql <= mid) res += ask(p << 1, l, mid, ql, qr, ql, qry);
112     if(mid < qr) res += ask(p << 1 | 1, mid + 1, r, ql, qr, ql, qry);
113     return res;
114 }
115 };

```

李超线段树

```

1 //普通李超线段树
2 //用于查询若干线段或者直线在某点处的最值
3 const double eps = 1e-9, INF = 1e18;
4
5
6 int sgn(double x){
7     if(fabs(x) < eps) return 0;
8     return x < 0 ? -1 : 1;
9 }
10
11 struct Line{
12     double k, b;
13     bool exists;
14
15     //更改处 (-INF)
16     Line(double k = 0, double b = INF) : k(k), b(b), exists(false) {}
17
18     double cal(int x)const{
19         return k * x + b;
20     }
21 };
22 //插入线段复杂度为 log * log 其余为 log (当前为维护最小值版本)
23 struct LCseg{
24     int n;
25     vector<Line> tr;
26     LCseg(int n) : n(n), tr(n * 4) {}
27
28     void add(int p, int l, int r, Line val){
29         if(!tr[p].exists){
30             tr[p] = val;
31             tr[p].exists = true;
32         }
33     }
34 };

```

```

32     }
33     int mid = l + r >> 1;
34     //更改处 <
35     if(tr[p].cal(mid) > val.cal(mid)) swap(tr[p], val);
36     if(l == r) return;
37     //更改处 >
38     if(val.cal(l) < tr[p].cal(l)){
39         add(p << 1, l, mid, val);
40     } else {
41         add(p << 1 | 1, mid + 1, r, val);
42     }
43 }
44
45 // 插入一条全局直线
46 void addLine(double k, double b){
47     Line l(k, b);
48     l.exists = true;
49     add(1, 1, n, l);
50 }
51
52 // 插入一条在 [L, R] 范围内的线段
53 void addSeg(int p, int l, int r, int ql, int qr, Line val){
54     if(ql <= l && r <= qr){
55         add(p, l, r, val);
56         return;
57     }
58     int mid = l + r >> 1;
59     if(ql <= mid) addSeg(p << 1, l, mid, ql, qr, val);
60     if(mid < qr) addSeg(p << 1 | 1, mid + 1, r, ql, qr, val);
61 }
62 // 查询 x 处的最大 y 值
63 double ask(int p, int l, int r, int x){
64     if(x < l || x > r || !tr[p].exists) return INF; //更改处-INF
65     double res = tr[p].cal(x);
66     if(l == r) return res;
67     int mid = l + r >> 1;
68     //更改处 max
69     if(x <= mid) return min(res, ask(p << 1, l, mid, x));
70     return min(res, ask(p << 1 | 1, mid + 1, r, x));
71 }
72 };
73
74
75 //离散化李超线段树（对所有可能的x值进行离散化处理）（最大值版本）
76 const int INF = 2e18; // 维护最大值，初始b设为-INF
77
78 struct Line {
79     int k, b;
80     bool exists;
81     Line(int k = 0, int b = -INF) : k(k), b(b), exists(false) {}
82     int cal(int x) const {
83         return k * x + b;
84     }
85 };
86
87 struct LCSeg {
88     int n;
89     vector<Line> tr;
90     vector<int>& coords; //离散化后的数组，sort+erase(unique)
91
92     LCSeg(int n, vector<int>& c) : n(n), tr(n * 4), coords(c) {}

```

```

93
94 void add(int p, int l, int r, Line val) {
95     if (!tr[p].exists) {
96         tr[p] = val; tr[p].exists = true;
97         return;
98     }
99
100     int mid = l + r >> 1;
101
102     int mid_x = coords[mid - 1];
103     int l_x = coords[l - 1];
104     int r_x = coords[r - 1];
105
106     if (tr[p].cal(mid_x) < val.cal(mid_x)) swap(tr[p], val);
107     if (l == r) return;
108
109     if (val.cal(l_x) > tr[p].cal(l_x)) add(p << 1, l, mid, val);
110     else if (val.cal(r_x) > tr[p].cal(r_x)) add(p << 1 | 1, mid + 1, r, val);
111 }
112
113 void addLine(int k, int b) {
114     Line l(k, b);
115     l.exists = true;
116     add(1, 1, n, l);
117 }
118
119 int ask(int p, int l, int r, int idx) {
120     if (!tr[p].exists) return -INF;
121     int cur_x = coords[idx - 1];
122     int res = tr[p].cal(cur_x);
123     if (l == r) return res;
124
125     int mid = l + r >> 1;
126     if (idx <= mid) return max(res, ask(p << 1, l, mid, idx));
127     return max(res, ask(p << 1 | 1, mid + 1, r, idx));
128 }
129 };
130
131 // ===== 分数版李超线段树 =====
132 // k、b 都可以是分数 p/q。查询点为整数，所有比较用 __int128 精确完成，没有浮点误差。
133 // 下面实现的是最小值版本；若需要最大值，把 add 和 ask 中标注"方向"处的比较符号反过来即可。
134 struct Frac {
135     __int128 p, q; // 表示 p/q, 始终 q > 0 且已约分
136
137     Frac(__int128 p = 0, __int128 q = 1) : p(p), q(q) { norm(); }
138
139     static __int128 gcd128(__int128 a, __int128 b) {
140         if (a < 0) a = -a;
141         if (b < 0) b = -b;
142         while (b) {
143             __int128 t = a % b;
144             a = b;
145             b = t;
146         }
147         return a;
148     }
149
150     void norm() {
151         if (q == 0) { // 实际使用中不会出现分母为 0
152             exit(1);
153         }

```

```

154         if (q < 0) {
155             p = -p;
156             q = -q;
157         }
158         if (p == 0) {
159             q = 1;
160             return;
161         }
162         __int128 g = gcd128(p, q);
163         p /= g;
164         q /= g;
165     }
166
167     Frac operator*(long long x) const { return Frac(p * x, q); }
168     Frac operator+(const Frac& o) const { return Frac(p * o.q + o.p * q, q *
o.q); }
169     Frac operator-(const Frac& o) const { return Frac(-p * o.q + o.p * q, q *
o.q); }
170
171     bool operator<(const Frac& o) const { return p * o.q < o.p * q; }
172     bool operator>(const Frac& o) const { return p * o.q > o.p * q; }
173     bool operator==(const Frac& o) const { return p == o.p && q == o.q; }
174     bool operator<=(const Frac& o) const { return !(*this > o); }
175     bool operator>=(const Frac& o) const { return !(*this < o); }
176
177     long long toLL() const { return (long long)(p / q); }
178     double toDouble() const { return (double)p / (double)q; }
179 };
180
181 struct LineFrac {
182     Frac k, b;
183     bool exists;
184     // 空节点初始化为一个足够大的值，用于最小值版本
185     // 【方向】 最小值用 +INF; 若改最大值，这里改为足够小的值，如 -(1LL << 62)
186     LineFrac(Frac k = 0, Frac b = Frac(1LL << 62, 1)) : k(k), b(b), exists(false)
{}
187
188     Frac cal(int x) const {
189         return k * x + b;
190     }
191 };
192
193 struct LCSegFrac {
194     int n;
195     vector<LineFrac> tr;
196
197     LCSegFrac(int n) : n(n), tr(n * 4) {}
198
199     void add(int p, int l, int r, LineFrac val) {
200         if (!tr[p].exists) {
201             tr[p] = val;
202             tr[p].exists = true;
203             return;
204         }
205         int mid = l + r >> 1;
206         // 最小值版本：中点更小/更优的留在当前节点
207         // 【方向】 最小值：当前中点值 > 新线中点值时交换（让更小的留在当前）
208         if (tr[p].cal(mid) > val.cal(mid)) swap(tr[p], val);
209         if (l == r) return;
210
211         // 【方向】 最小值：新线在左端点更小则往左走（改最大值时改为 >）
212         if (val.cal(l) < tr[p].cal(l)) {

```



```

213         add(p << 1, 1, mid, val);
214         // 【方向】 最小值: 新线在右端点更小则往右走 (改最大值时改为 >)
215     } else if (val.cal(r) < tr[p].cal(r)) {
216         add(p << 1 | 1, mid + 1, r, val);
217     }
218 }
219
220 // 插入全局直线  $y = k*x + b$ ,  $k$  和  $b$  都可以是分数
221 void addLine(const Frac& k, const Frac& b) {
222     LineFrac l(k, b);
223     l.exists = true;
224     add(1, 1, n, l);
225 }
226
227 // 直接传  $p/q$  形式:  $y = (p/q)*x + (u/v)$ 
228 void addLine(long long p, long long q, long long u, long long v) {
229     addLine(Frac(p, q), Frac(u, v));
230 }
231
232 // 插入一条在  $[L,R]$  范围内的线段
233 void addSeg(int p, int l, int r, int ql, int qr, LineFrac val) {
234     if (ql <= l && r <= qr) {
235         add(p, l, r, val);
236         return;
237     }
238     int mid = l + r >> 1;
239     if (ql <= mid) addSeg(p << 1, l, mid, ql, qr, val);
240     if (mid < qr) addSeg(p << 1 | 1, mid + 1, r, ql, qr, val);
241 }
242
243 void addSeg(int ql, int qr, const Frac& k, const Frac& b) {
244     LineFrac l(k, b);
245     l.exists = true;
246     addSeg(1, 1, n, ql, qr, l);
247 }
248
249 Frac ask(int p, int l, int r, int x) {
250     // 【方向】 最小值: 空节点返回  $+\text{INF}$  (改最大值时改为  $-\text{INF}$ )
251     if (x < l || x > r) return Frac(1LL << 62, 1);
252     Frac res = tr[p].exists ? tr[p].cal(x) : Frac(1LL << 62, 1);
253     if (l == r) return res;
254     int mid = l + r >> 1;
255     // 【方向】 最小值: 合并时取  $\min$  (改最大值时改为  $\max$ )
256     if (x <= mid) return min(res, ask(p << 1, l, mid, x));
257     return min(res, ask(p << 1 | 1, mid + 1, r, x));
258 }
259
260 Frac ask(int x) {
261     return ask(1, 1, n, x);
262 }
263 };

```

主席树

```

1 //https://www.luogu.com.cn/problem/P3919
2 //主席树单点修改单点查询
3
4 const int N=1e6+5;
5
6 struct node{

```

```

7     int l,r;
8     long long val;
9 }tree[N*40];
10 //模版上写20,但是我交了re,写大一点过了,有些玄学
11
12 vector<long long> vec(N);
13 vector<int> root(N); //root表示第i个版本的根节点的index
14
15 int top=0;
16 //在更新操作时进行动态开点的函数
17 int clone(int ind){
18     top++;
19     tree[top]=tree[ind];
20     return top;
21 }
22
23 //建树
24 int build(int ind,int l,int r){
25     top++;
26     ind=top;
27
28     if(l==r){
29         tree[ind].val=vec[l];
30         return ind;
31     }
32
33     int mid=(l+r)/2;
34     //实际上更新点可以有很多种方式,我使用的方式是为了统一更新建树和查询函数参数的格式
35     //build里面的第一个参数意义不大,为了统一参数那么写的,写一样的就行
36     tree[ind].l = build(tree[ind].l,l,mid);
37     tree[ind].r = build(tree[ind].r,mid+1,r);
38     return ind;
39 }
40
41 //更新
42 int update(int ind,int l,int r,int t,long long v){
43     ind=clone(ind);
44
45     if(l==r){
46         tree[ind].val=v;
47         return ind;
48     }
49     int mid=(l+r)/2;
50     if(t<=mid){
51         tree[ind].l=update(tree[ind].l,l,mid,t,v);
52     }else{
53         tree[ind].r=update(tree[ind].r,mid+1,r,t,v);
54     }
55     return ind;
56 }
57
58 //查询
59 long long query(int ind,int l,int r,int t){
60     if(l==r){
61         return tree[ind].val;
62     }
63
64     int mid=(l+r)/2;
65     if(t<=mid){
66         return query(tree[ind].l,l,mid,t);
67     }else{

```

```

68         return query(tree[ind].r,mid+1,r,t);
69     }
70 }

```

并查集

```

1  //并查集
2  struct SDU{
3      int n, cnt;//cnt为连通块数量
4      vector<int> fa;
5      SDU(int m = 0){
6          n = m;
7          fa.resize(n + 1);
8          for(int i = 1; i <= n; i++) fa[i] = i;//1-based
9          cnt = n;
10     }
11
12     int find(int x){
13         return x == fa[x] ? x : fa[x] = find(fa[x]);
14     }
15
16     void merge(int x, int y){
17         int xx = find(x);
18         int yy = find(y);
19         if(xx != yy){
20             fa[xx] = yy;
21             cnt--;
22         }
23     }
24
25     bool check(int x, int y){
26         return find(x) == find(y);
27     }
28 };

```

带权并查集

```

1  //带权并查集，维护每个连通块根节点到每个节点中某个权值关系
2  struct DSU{
3      int n;
4      vector<int> fa, sz, dis;
5
6      DSU(int n):n(n),fa(n+1),sz(n+1,1),dis(n+1,0){
7          for(int i=1;i<=n;i++){
8              fa[i]=i;
9          }
10     }
11
12     int find(int x){
13         if(fa[x]==x) return x;
14
15         int f=fa[x];
16         fa[x]=find(fa[x]);
17
18         dis[x]+=dis[f];
19
20         return fa[x];
21     }
22

```

```

23     bool merge(int x,int y,int d){
24         int fx=find(x);
25         int fy=find(y);
26
27         if(fx==fy){
28             return dis[y]-dis[x]==d;
29         }
30
31         if(sz[fx]>sz[fy]){
32             swap(x,y);
33             swap(fx,fy);
34             d=-d;
35         }
36
37         fa[fx]=fy;
38
39         dis[fx]=dis[y]-dis[x]-d;
40
41         sz[fy]+=sz[fx];
42
43         return true;
44     }
45 };

```

普通莫队

```

1  #define int long long
2  #define double long double
3  const int maxn = 5e4;
4  int B;
5  int cur = 0;
6
7  struct Q{
8      int l, r, id;
9
10     bool operator<(const Q& b)const{
11         int pos1 = l / B, pos2 = b.l / B;
12         if(pos1 != pos2) return pos1 < pos2;
13         if(pos1 & 1) return r < b.r;
14         return r > b.r;
15     }
16 };
17
18 vector<int> cnt(maxn + 5);
19 vector<int> a(maxn + 5);
20 vector<Q> ask(maxn + 5);
21 vector<pair<int,int>> ans(maxn + 5);
22
23 int l = 1, r = 0;
24
25 void add(int pos){
26     int val = a[pos];
27     cur += cnt[val];
28     cnt[val]++;
29 }
30
31 void del(int pos){
32     int val = a[pos];
33     cnt[val]--;
34     cur -= cnt[val];

```

```

35 }
36
37 void solve(){
38     int n, m;
39     cin >> n >> m;
40     B = max(1.0, n / sqrt(m * 2.0 / 3.0));
41     for(int i = 1; i <= n; i++){
42         cin >> a[i];
43     }
44     for(int i = 0; i < m; i++){
45         cin >> ask[i].l >> ask[i].r;
46         ask[i].id = i;
47     }
48     sort(ask.begin(), ask.begin() + m);
49     for(int i = 0; i < m; i++){
50         auto[q1, qr, id] = ask[i];
51         while(l > q1) add(--l);
52         while(r < qr) add(++r);
53         while(l < q1) del(l++);
54         while(r > qr) del(r--);
55         int len = r - l + 1;
56         int fm = cur == 0 ? 1 : len * (len - 1) / 2;
57         int g = cur == 0 ? 1 : __gcd(cur, fm);
58         ans[id] = {cur / g, fm / g};
59     }
60     for(int i = 0; i < m; i++){
61         cout << ans[i].first << '/' << ans[i].second << '\n';
62     }
63 }
64

```

帶修莫隊

```

1  #define int long long
2  #define double long double
3  int n, m, B;
4  struct Q{
5      int l, r, t, id;
6
7      bool operator<(const Q& b) const{
8          if(l / B != b.l / B) return l / B < b.l / B;
9          if(r / B != b.r / B) return r / B < b.r / B;
10         return t < b.t;
11     }
12
13     Q(int l, int r, int t, int id) : l(l), r(r), t(t), id(id) {}
14 };
15
16 struct OP{
17     int p, x;
18
19     OP(int p, int x) : p(p), x(x) {}
20 };
21
22 vector<int> a;
23 vector<Q> ask;
24 vector<OP> op;
25 vector<int> cnt(1000006);
26 int res = 0;
27

```

```

28 void add(int val){
29     if(cnt[val] == 0) res++;
30     cnt[val]++;
31 }
32
33 void del(int val){
34     cnt[val]--;
35     if(cnt[val] == 0) res--;
36 }
37
38 void solve(){
39     cin >> n >> m;
40     a.resize(n);
41     for(int i = 0; i < n; i++){
42         cin >> a[i];
43     }
44
45     for(int i = 0; i < m; i++){
46         char c;
47         int x, y;
48         cin >> c >> x >> y;
49         if(c == 'Q'){
50             ask.push_back(Q(x - 1, y - 1, op.size(), ask.size()));
51         } else {
52             op.push_back(OP(x - 1, y));
53         }
54     }
55     B = pow(n, 2.0 / 3.0);
56     sort(ask.begin(), ask.end());
57     vector<int> ans(ask.size());
58     int curT = 0, l = 0, r = -1;
59     for(int i = 0; i < ask.size(); i++){
60         auto[q1, qr, t, id] = ask[i];
61         while(l > q1) add(a[--l]);
62         while(r < qr) add(a[++r]);
63         while(l < q1) del(a[l++]);
64         while(r > qr) del(a[r--]);
65         while(curT < t){
66             int pos = op[curT].p;
67             if(l <= pos && pos <= r){
68                 del(a[pos]), add(op[curT].x);
69             }
70             swap(a[pos], op[curT].x);
71             curT++;
72         }
73         while(curT > t){
74             curT--;
75             int pos = op[curT].p;
76             if(l <= pos && pos <= r){
77                 del(a[pos]), add(op[curT].x);
78             }
79             swap(a[pos], op[curT].x);
80         }
81         ans[id] = res;
82     }
83     for(int i : ans) cout << i << '\n';
84 }
85

```

```

1  #define int long long
2  #define double long double
3  int n, q, B;
4  struct Q{
5      int l, r, id;
6
7      bool operator<(const Q& b)const{
8          if(l / B != b.l / B) return l / B < b.l / B;
9          return r < b.r;
10     }
11     Q(int l, int r, int id) : l(l), r(r), id(id) {}
12 };
13
14 vector<int> a;
15 vector<Q> ask;
16 vector<int> fst, lst;
17 vector<int> cnt(200005);
18
19 void solve(){
20     cin >> n;
21     a.resize(n);
22     fst.resize(n + 5, -1);
23     lst.resize(n + 5, -1);
24     for(int i = 0; i < n; i++){
25         cin >> a[i];
26     }
27     cin >> q;
28     B = max(1.0, n / sqrt(q * 2.0 / 3.0));
29     auto b = a;
30     sort(b.begin(), b.end());
31     b.erase(unique(b.begin(), b.end()), b.end());
32     for(int i = 0; i < n; i++){
33         a[i] = lower_bound(b.begin(), b.end(), a[i]) - b.begin();
34     }
35     for(int i = 0; i < q; i++){
36         int x, y;
37         cin >> x >> y;
38         ask.push_back(Q(x - 1, y - 1, i));
39     }
40     sort(ask.begin(), ask.end());
41     vector<int> ans(q);
42     int lb = -1, r = -1, res = 0;
43     for(int i = 0; i < q; i++){
44         auto[q1, qr, id] = ask[i];
45         int cur = q1 / B;
46
47         if(cur != lb){
48             fill(fst.begin(), fst.end(), -1);
49             fill(lst.begin(), lst.end(), -1);
50             res = 0;
51             lb = cur;
52             r = (cur + 1) * B - 1;
53         }
54
55         if(cur == qr / B){
56             int temp = 0;
57             vector<int> wait;
58             for(int j = q1; j <= qr; j++){
59                 int val = a[j];
60                 if(fst[val] == -1){
61                     fst[val] = j;

```

```

62         wait.push_back(val);
63     }
64     lst[val] = j;
65     temp = max(temp, j - fst[val]);
66 }
67 ans[id] = temp;
68 for(int val : wait){
69     fst[val] = -1;
70     lst[val] = -1;
71 }
72 continue;
73 }
74
75
76 while(r < qr){
77     r++;
78     int val = a[r];
79     if(fst[val] == -1) fst[val] = r;
80     lst[val] = r;
81     res = max(res, lst[val] - fst[val]);
82 }
83
84 int l = min(n, (cur + 1) * B);
85 int tmp = res;
86 stack<array<int,3>> wait;// val fst lst
87 while(l > ql){
88     l--;
89     int val = a[l];
90     wait.push({val, fst[val], lst[val]});
91     if(lst[val] == -1) lst[val] = l;
92     fst[val] = l;
93     res = max(res, lst[val] - fst[val]);
94 }
95
96 ans[id] = res;
97 res = tmp;
98
99 while(wait.size()){
100     auto[val, l, r] = wait.top();
101     wait.pop();
102     fst[val] = l;
103     lst[val] = r;
104 }
105
106 }
107
108 for(int i : ans) cout << i << '\n';
109
110 }
111

```